

Lab 6 - Generative Models

Name: Michael Opoku Mbroh

Student ID: 114202103

China Medical University

INTRODUCTION

Generative models enables the synthesis of realistic images, audio and other complex data distributions. Denoising diffusion probabilistic models (DDPMs) which is a generative approach helps by learning to reverse a gradual noising process to produce high quality samples.

In this lab, I implemented a conditional DDPM capable of generating synthetic images of 3D objects from the i-CLEVR dataset. Each condition specifies a set of objects defined by their color and shape and the model must generate a 64x64 image containing all the specified objects simultaneously.

My overall pipeline consists of four steps which are dataset and data loader for the i-CLEVR training images, conditional UNet model with time and label embeddings, training loop and a sampling pipeline that performs the reverse diffusion process to generate images conditioned on test labels. Generated images are evaluated using a pretrained ResNet18 classifier.

IMPLEMENTATION DETAILS

Dataset and Data Loader

The i-CLEVR dataset contains 18,009 training images each paired with a list of one to three object labels. Each label describes an object by its color and shape drawn from 8 colors x 3 shapes = 24 object classes as defines in objects.json.

Label Encoding

Each condition is encoded as a 24-dimensional one hot vector. For an image containing multiple objects, the corresponding indices are all set to 1. The function labels_to_onehot() handles this encoding. `vec = torch.zeros(24); vec[obj2idx[obj]] = 1.0` for obj in object list

Image Preprocessing

All training images are resized to 64x64 pixels using a Resize + CenterCrop pipeline. Pixel values are then normalized to the range [-1,1] using the standard torchvision normalization. The dataloader uses `drop_last=True` to ensure constant batch sizes during training and `pin_memory=True` for efficient GPU data transfer.

Transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5]).

Model Architecture

The backbone of the conditional DDPM is a UNet with residual blocks, self-attention at the bottleneck and separate embedding paths for the timestep and the condition label. The architecture used is from (Ho et al., 2020) and the hugging face diffusion model tutorial.

UNet Structure

The UNet operates on 64x64 images and follows an encoder bottleneck decoder structure

Stage	Channels	Resolution	Components
Encoder level 0	64	64x64	2x ResBlock + Conv stride 2
Encoder level 1	128	32x32	2x ResBlock + Conv stride 2
Encoder level 2	256	16x16	2x ResBlock (no downsample)
Encoder level 3	256	16x16	2x ResBlock
Bottleneck	256	16x16	ResBlock + Attention + ResBlock
Decoder	256 to 64	16 to 64	2x ResBlock + ConvTranspose2d

Residual Block (ResBlock)

Each ResBlock applies GroupNorm(8) – SiLu – Conv2d(3x3) – time projection - GroupNorm(8) – SiLu – Conv2d(3x3) with a residual skip connection. The time embedding is injected by projecting it to the output channel size and adding it to the intermediate feature map.

```
h = conv1 (silu(norm1(x))); h = h + t_proj (t_emb)[:, :, None, None]; h=conv2(silu(norm2(h)))
```

Self-Attention

Self-attention is applied exclusively at the bottleneck with 4 heads. The spatial feature map is reshaped from (B, C, H, W) to (B, HxW, C) before the attention operation, then reshaped back. Restricting attention to the bottleneck reduces memory usage making training feasible on a 6GB GPU.

Time Embedding

Timestep t is encoded using sinusoidal positional embedding of dimension 128. The sinusoidal embedding is passed through a 2-layer MLP with SiLU activation to produce the final time embedding of dimension 128.

Condition Embedding

The 24-dimensional one-hot label vector is projected through a two-layer MLP to produce a condition embedding of the same dimension as the time embedding. The two embeddings are added together. This combined embedding ensures the condition influences the denoising at every scale.

`emb = t_proj (get_time_embedding(t, 128)) + cond_proj (cond)`

Model Configuration

Hyperparameter	Value
Image channels	3 (RGB)
Image size	64x64
Down channels	[64, 128, 256, 256]
Time embedding dim	128
Attention heads	4
Number of classes	24

Noise Schedule

I used the standard linear beta schedule from Ho et al., (2020).

`betas = torch.linspace(1e-4, 0.02, 1000)`

From the betas, I compute the cumulative products which defines how much signal is preserved at each timestep. The `linearNoiseScheduler` class precomputes $\sqrt{\bar{\alpha}_t}$ and $\sqrt{1-\bar{\alpha}_t}$ for all T timesteps.

Training Procedure

Loss Function

I used the epsilon parameterization loss from Ho, J. et al., (2020). The UNet is trained to predict the noise that was added during the forward process, and the loss is the mean squared error between the predicted and actual noise.

Gradient Accumulation and Exponential Moving Average (EMA)

Due to my 6GB VRAM constraint I used a physical batch size of 4 with gradient accumulation over 8 steps giving an effective batch size of 32.

I maintained an EMA of the model weights with decay = 0.9999. After each optimizer step, the EMA shadow weights are updated.

Training Hyperparameters

Optimizer	Adam
Learning rate	1e-4
Physical batch size	4
Gradient accumulation steps	8
Total epochs	150
EMA decay	0.9999
Diffusion timesteps	1000
B start / β end	0.0001 / 0.02

Sampling and Evaluation

At inference time, I start from pure Gaussian noise $x_T \sim N(0,1)$ and iteratively denoise for T steps. At each step t, the model predicts the noise and I compute the previous sample $x_{\{t-1\}}$ using the DDPM reverse posterior. Without the $x0_pred$ clamping steps values can diverge and produce entirely black images.

Generated images were evaluated using a pretrained ResNet18 classifier. The evaluator was trained on the i-CLEVR dataset and classifies 24 object types. For each generated image it predicts the top-k object classes and computed accuracy by counting how many predicted classes match the ground truth labels. My generated images was normalized with $\text{Normalize}((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))$ which corresponds to pixel values $[-1, 1]$.

RESULTS AND DISCUSSION

After 150 epochs of training and evaluation on both test files I got an accuracy of 0.7361 on test.json file and 0.8810 on new_test.json file.

Accuracy Screenshot

```
[20]: acc_test, acc_new = run_all(model, scheduler)

=====
Lab 6 | Inference + Evaluation
=====

Generating 32 images [test.json]
  Saved 32 PNGs -> ./images/test/
  Saved grid -> ./images/test/grid.png
  >>> test.json accuracy : 0.7361

Generating 32 images [new_test.json]
  Saved 32 PNGs -> ./images/new_test/
  Saved grid -> ./images/new_test/grid.png
  >>> new_test.json accuracy : 0.8810

Denoising process: ['red sphere', 'cyan cylinder', 'cyan cube']
  Frames: 10 (left=noisy right=clean)
  Saved -> ./images/denoising_process.png

=====
FINAL RESULTS
=====
  test.json : 0.7361
  new_test.json : 0.8810
=====

Files saved:
  images/test/0.png ... 31.png
  images/test/grid.png
  images/new_test/0.png ... 31.png
  images/new_test/grid.png
  images/denoising_process.png
```

Figure 1: accuracy evaluation output for test.json and new_test.json

Synthetic Image Grids

The following grids show the 32 generated images for each test file, arranged in 8 columns x 4 rows using `torchvision.utils.make_grid(images, nrow=8)`. Each image is conditioned on the corresponding label set from the test.json file. The generated images show 64x64 renders of colored 3D objects on a gray background.



Figure 2: Synthetic image grid for test.json

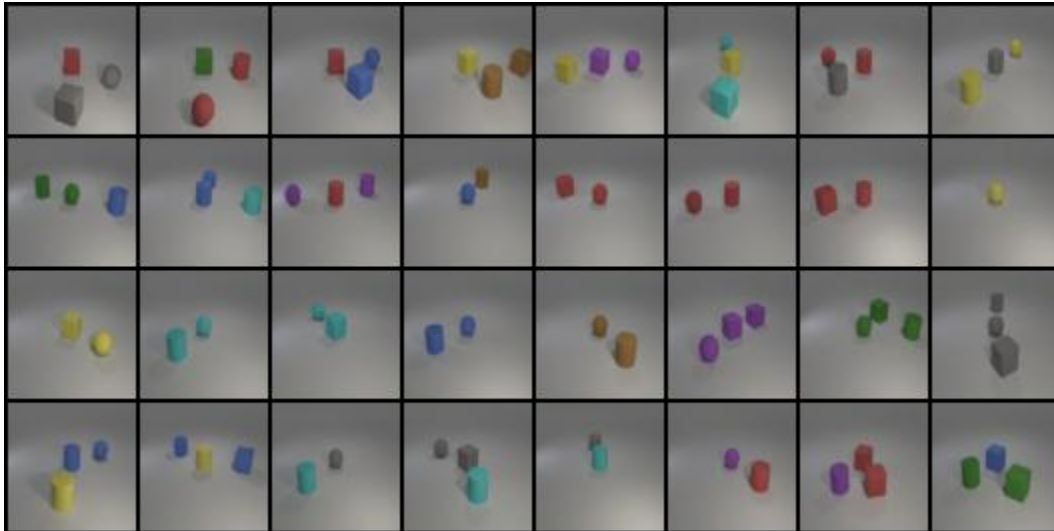


Figure 3: Synthetic image grid for new_test.json

Denoising Process Visualization

Figure 4 shows the denoising process for the label set [“red sphere”, “cyan cylinder”, “cyan cube”]. The visualization shows progressive emergence where early frames appears as pure noise, intermediate frames show blurry color regions forming and the final frame shows clearly colored 3D objects matching the specified condition.



Figure 4: Denoising process for [“red sphere”, “cyan cylinder”, “cyan cube”] – left=noisy, right=clean

REFERENCES

Dhariwal, P., & Nichol, A (2021). Diffusion models beat GANs on image synthesis. NeurlPS

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. NeurlPS

Hugging face. Diffusion models course.

Nichol, A., & Dhariwal, P. (2021). Improved denoising diffusion models. ICML.